Skribix

Hand-drawn Sketch Recognition Project

Sahil Narkhede, Sarah Fatima, Anshit Agarwal,
Krish Teckchandani, Yashraj Rathod

Indian Institute of Technology, Jodhpur
{b23cs1060,b23ee1093,b23cs1087,b23cs1092,b23cs1056}@iitj.ac.in

April 14, 2025

Abstract

Recognizing hand-drawn sketches presents a unique and challenging problem in the field of computer vision due to the abstract, sparse, and highly variable nature of sketches compared to natural images. Unlike photographs, sketches often lack texture, shading, and fine detail, relying instead on a few key strokes to convey the semantic content of an object. This project aims to address the problem of recognizing such sketches by building a robust sketch classification pipeline based on low-level visual feature extraction and the Bag of Visual Words (BoVW) model.

To approach this, we utilized a publicly available crowd-sourced dataset initially introduced in the paper *How Do Humans Sketch Objects?*. The full dataset comprises 20,000 sketches across 250 object categories.

For our implementation, we considered two versions of the dataset, based on their size. The first included all 250 categories, while the second was a focused subset consisting of 15 semantically diverse categories such as Airplane, Book, Pizza, and Traffic Light. This was done in order to improve accuracy, prevent misclassification, and make it computationally convenient. This subset, comprising 1,200 images, served as the primary dataset for model training and evaluation in our experiments. After feature extraction, the global feature vectors and their corresponding class labels were stored as NumPy arrays (`image_features.npy` and `image_labels.npy`). In the version-1 dataset, 20,000 images made our classification task very difficult.

We integrated Real-Time Prediction Integration which consists of a complete end-to-end prediction system. It consists of a flask-based canvas where the user can sketch their model. Following this, after pressing the predict button, the model outputs prediction based on our best possible model CNNs, which was mostly used for analysis. This enables us to see our model work in a practical landscape. We have utilised Google-Cloud for the same.

In conclusion, this project demonstrates a scalable and interpretable approach to sketch recognition by employing various feature extraction techniques, machine learning methodologies and real-time implementation. This project offers foundational results for further research in both human sketch analysis and minimalist object recognition tasks.

Keywords: Sketch Recognition, Bag of Visual Words, Feature Extraction, Machine Learning techniques, Hand-drawn Sketches

Figure 1: *In this paper we explore how machines recognize hand-drawn sketches*

Contents

1 Introduction

Sketch-based image recognition poses a unique challenge in the field of computer vision due to the abstract, sparse, and stylized nature of hand-drawn sketches. Unlike natural photographs, sketches often lack texture, color, and shading—relying entirely on simple strokes to convey semantic meaning. In this project, we aim to address the problem of identifying hand-drawn object sketches using an end-to-end classification pipeline.

1.1 Citing papers

We used the siggraph 2012 classify-sketch dataset introduced in [1] for training and evaluation.
We also referred to [5] for the CNN model architecture implemented in the working prototype of Skribix.
We have used LLMs [7] for specific model implementations and optimizations.
More miscellaneous references used [8]

1.2 Problem Statement

The primary objective of this work is to build a robust system capable of recognizing hand-drawn sketches of objects, even in the absence of color or fine details.It basically focuses on sketches sketched by non-professionals. The pipeline focuses on low-level visual feature extraction followed by implementation of various machine learning techniques to compare and contrast the results to draw appropriate results accordingly. The end goal is to accurately classify unseen sketches into their corresponding object categories.

1.3 Dataset Details

We used a publicly available dataset originally introduced in the paper *How Do Humans Sketch Objects?*. This dataset contains a diverse collection of 20,000 hand-drawn sketches covering 250 distinct categories, with each category reflecting everyday functional objects.

Collection Process:

- **Source:** Amazon Mechanical Turk (AMT) crowd-sourcing platform.
- **Participants:** 1,350 unique contributors, spending a total of 741 hours.
- **Instructions:** Each participant was asked to sketch objects using a stroke-based canvas with options like undo, redo, clear, and delete. They were instructed to focus on outlines without adding excessive contextual elements.
- **Sketch Characteristics:** Median drawing time per sketch was 86 seconds, with a median of 13 strokes. Most sketches followed a coarse-to-fine drawing strategy, starting with longer strokes.
- **Quality Control:** Sketches were manually reviewed. Around 6.3% of entries were removed for being offensive, incorrect, or out-of-category. The dataset was then balanced to contain exactly 80 sketches per category.

Subset Selection for Our Work: For implementation, we used two versions of the dataset:

- **Full Dataset:** All 250 categories totaling 20,000 images.
- **Subset:** A selected subset of 15 semantically diverse categories for focused experimentation. These include: Airplane, Book, Cup, Envelope, Fan, Fork, Hat, Key, Laptop, Leaf, Moon, Pizza, T-shirt, Traffic Light, and Wineglass.

1.4 Major Findings

Best Overall Performance

Version 2.3's K-Means + SVM achieved the highest testing accuracy of **84.16%**, highlighting the effectiveness of edge-based features when paired with structured classification techniques like SVM.

Most Balanced Model

Version 2.1's KMeans (k=45) + SVM offered a well-balanced trade-off between model complexity and generalization, achieving a testing accuracy of **81.25%** without overfitting.

Most Promising Deep Learning Approach

Version 2.4's CNN yielded a promising testing accuracy of **73.67%**, indicating the capability of deep learning models to handle sketch-based data, with significant room for improvement given more training data and augmentation.

Impact of Class Reduction:

Reducing the number of classes from 250 to 15 drastically improved training efficiency and recognition accuracy across all models. Simpler class boundaries made learning more effective.

Surprising Effectiveness of Edge Features:

Edge detection in Version 2.3 emerged as a highly competitive feature extraction method, outperforming more complex pipelines in terms of accuracy and robustness.

SVM Consistency Across Versions:

Support Vector Machines (SVM) consistently outperformed other classifiers when paired with appropriate feature representations, such as PCA-reduced features or cluster-based histograms.

Potential of Deep Learning:

CNNs provided strong initial results and showed significant potential for scalability. However, their full capacity remains untapped due to the limited dataset size in our current setup.

1.5 Methodology Overview

The approaches tried are distributed over two sections- version 1 and version-2. Version-1 has the entire 250 class dataset, it has several models trained in the initial stage of the project. Version-2 consists of models trained over chosen 15 class dataset. It has improved accuracy, lesser misclassification, and better results. Refer to the appendix to have an overview of models used.

Comparison of Models Used in Version 1 and Version 2		
Version 1: 250 classes × 80 images — Version 2: 15 classes		
S.No.	**Version 1 Models**	**Version 2 Models**
1	PCA + Naive Bayes	PCA + Naive Bayes
2	PCA + KNN	PCA + KNN (k=3)
3	K-Means + KNN	K-Means (k=40) + SVM
4	Mean Shift Clustering + KNN	GMM + KNN
5	SVM	ANN
6	Mean Shift Clustering + SVM	SVM
7	ANN	CNN
8	GMM + KNN	PCA + SVM

Table 1: Models Used in Version 1 and Version 2

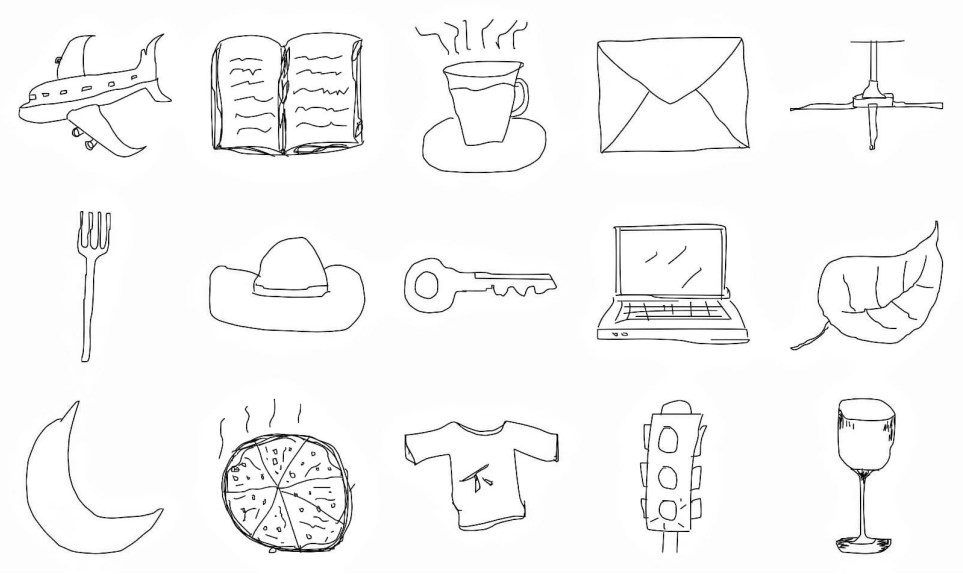

Figure 2: All the classes of version-2 dataset

The 15 Classes:

airplane	book	cup	envelope	fan
fork	hat	key	laptop	leaf
moon	pizza	t-shirt	traffic light	wineglass

2 Feature Extraction

We used multiple methods in version-2 of out dataset for feature extraction. They are listed as follows:

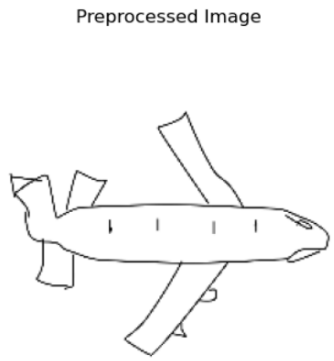

Figure 3: *Pre-processed illustration of original image*

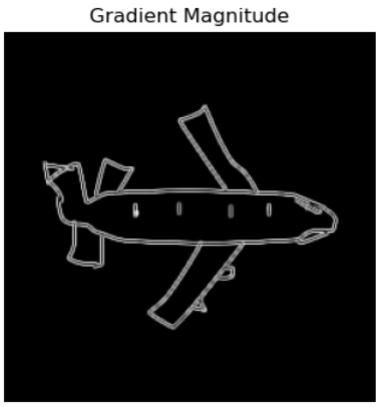

Figure 4: *Gradient-based feature visualization*

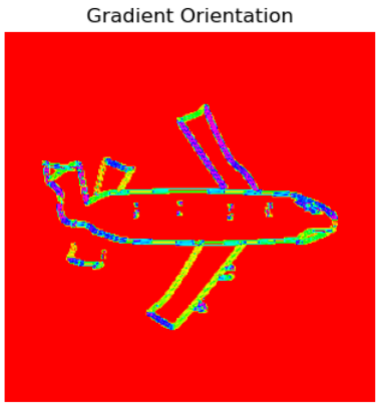

Figure 5: *Gradient orientation visualization*

Figure 6: Illustration of various stages of gradient computation process

2.1 sHOG:Sparse Histogram of Oriented Gradients

1. Image Preprocessing and Local Descriptor Extraction

The first phase of the feature extraction is preprocessing. Each input image is converted to grayscale. Images are then resized to a standard resolution of 256×256 pixels, ensuring uniformity across the dataset. Sobel filters are applied to compute the horizontal and vertical gradients, from which gradient magnitude and orientation are derived. Orientations are mapped to the unsigned range $[0, \pi)$ to align with sketch characteristics.This brings it in a similar range. These gradients provide localized edge information. Each image is further partitioned into 28×28 patches, where each patch is subdivided into a 4×4 grid to compute orientation histograms. The result is a 64-dimensional descriptor per patch, normalized.

2. Visual Vocabulary Creation Using K-Means

After descriptor extraction, a visual vocabulary is built to represent common patterns found across the dataset. All patch-level descriptors from each image are gathered into a single dataset. Each image's file path and class label are also recorded for supervised learning tasks. To create the vocabulary, K-Means clustering is applied to the combined descriptor pool. Each cluster center forms a "visual word", capturing frequently occurring local structures across sketches. The total number of clusters—set as the vocabulary size (e.g., 500)—determines the resolution of this representation. The final vocabulary is stored as a NumPy file (`vocabulary.npy`) for efficient reuse in later encoding steps.

3. Global Image Feature Extraction using BoVW

With the vocabulary in place, each image's local descriptors are encoded by assigning them to the nearest visual word using Euclidean distance. This process transforms the raw local descriptors into a histogram representing the frequency of each visual word in the image. These histograms capture the spatial occurrence of patterns, offering a fixed-length, global feature vector regardless of image complexity. For this setup, each image results in a 500-dimensional feature vector. By repeating this process for every image in the dataset, we obtain a consistent and compact representation suitable for classification tasks. This step bridges the gap between local patterns and holistic image understanding.

4. Feature Storage and Visualization

The final stage ensures that the extracted features and labels are properly stored and interpreted. The feature vectors are saved in `image_features.npy`, while class labels are stored in `image_labels.npy`. For verification and qualitative analysis, representative images from each class are visualized by printing their BoVW histograms. This visualization offers insight into inter-class variability and helps determine whether certain classes are more distinguishable than others. In total, 940,800 descriptors were extracted from 1,200 images, ultimately transformed into structured (1200, 500) feature matrices ready for downstream machine learning tasks.

2.2 HOG using Soft gradients

1. Image Preprocessing and Gradient Softening

The feature extraction process starts by standardizing sketch images by resizing them to a fixed resolution. This step simplifies computation, focusing solely on the structure of the sketch. After this, gradient computation is performed using Sobel filters to obtain horizontal and vertical edge responses. What sets this approach apart is the soft gradient quantization process, where instead of assigning a pixel's orientation to a single bin, it softly contributes to multiple orientation bins using Gaussian weighting. This "soft" mapping helps capture ambiguous or noisy edge orientations, often encountered in hand-drawn sketches, improving the robustness of descriptors to slight variations in stroke direction and noise.

2. Patch-Wise Descriptor Construction

Once soft gradient orientation responses are calculated for the entire image, the image is divided into non-overlapping patches, for example in a 28×28 grid layout. Each patch is treated as an independent local region from which texture information is extracted. Inside each patch, gradient magnitudes and their soft-assigned orientation contributions are pooled into histograms, usually across a fixed number of bins (e.g., 8). These soft histograms reflect the distribution of edge directions in a more continuous and stable manner than hard quantization. All histograms from a patch are then concatenated and optionally normalized to yield a robust local descriptor, capturing both edge strength and smooth orientation variation.

3. Feature Aggregation and Global Representation

After computing local descriptors for every patch, the final image representation is constructed by aggregating these descriptors into a single high-dimensional vector. The resulting vector captures fine-grained local textures while preserving the global layout of the sketch. Depending on the downstream task, this vector may be directly used as input for machine learning models or further reduced in dimensionality using techniques like PCA. This feature vector replaces the original pixel data with a compact, informative structure well-suited for sketch classification, especially in datasets where image variance arises from free-hand drawing styles.

4. Advantages and Applications

The soft gradient histogram approach excels in handling the uncertain and imprecise nature of sketches. By softly distributing gradient responses across orientation bins, it avoids abrupt decisions that can be sensitive to stroke noise or jitter. This results in more stable and discriminative features for downstream tasks such as KNN, SVM, or clustering algorithms. Additionally, the method is computationally lightweight and interpretable, making it suitable for scenarios where high-performance deep learning methods are infeasible or when a transparent feature representation is desired. Its combination of local detail preservation and robustness to sketch variability makes it a strong candidate for sketch recognition pipelines.

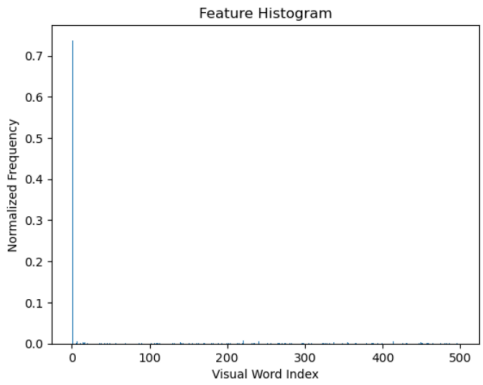

Figure 7: *Hard-assignment Histogram*

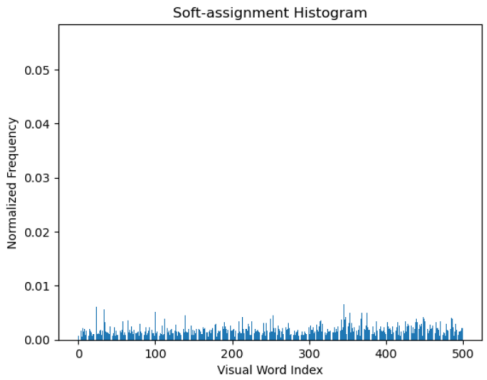

Figure 8: *Soft-assignment Histogram*

Figure 9: Comparison of hard & soft descriptor histograms

2.3 Edge Detection Features

1. Image Preprocessing and Edge Detection

This pipeline begins with a basic yet crucial preprocessing step: converting all images to grayscale and resizing them to a fixed resolution (e.g., 256×256). This ensures uniformity across the dataset and removes color dependencies, making it more suitable for sketch analysis. The core of this stage is edge detection, where Sobel or Canny edge detectors are applied to extract prominent contours from sketches. These methods highlight areas of high spatial gradient—typically the outlines and structural lines that define sketch content. The result is a binary or gradient magnitude map that clearly emphasizes object boundaries while discarding flat or low-contrast regions, allowing the feature extraction process to focus purely on structure.

2. Patch-Based Grid Sampling and Edge Pooling

Once the edge map is computed, the image is divided into a grid of non-overlapping patches (e.g., 28×28). Each patch is independently analyzed to summarize the strength and orientation of edges within its bounds. Unlike histogram-based descriptors, this method may rely on simpler statistics such as average edge magnitude, directionality, or count of edge pixels in each patch. These features are pooled together—typically through summation or averaging—forming a compact descriptor for that specific spatial region. This patch-wise analysis ensures that local edge structures are preserved, allowing the system to detect repeating geometric patterns or distinctive stroke arrangements.

3. Global Feature Vector Formation

After local features are extracted from each patch, they are flattened and concatenated to create a single global feature vector for each image. This vector represents the spatial distribution and strength of edge responses across the entire sketch. Because edges capture the primary structure of a drawing, this representation is highly informative for distinguishing classes in tasks such as sketch classification. The resulting vectors are of relatively low dimensionality compared to pixel-based or deep CNN features, making them lightweight and efficient for classical machine learning algorithms like SVM or KNN.

4. Usefulness and Application Scenarios

Edge-based features are particularly effective in sketch-based domains because sketches rely heavily on contours rather than texture or color. These descriptors are resilient to minor variations in stroke thickness, shading, or drawing style—common in hand-drawn sketches. Moreover, they are computationally efficient and interpretable, allowing quick prototyping and visualization of feature behavior. This method is ideal for small to medium-sized datasets where complex deep learning pipelines may not be feasible or necessary. When paired with dimensionality reduction or clustering, edge-based features can also facilitate visualization and analysis of large sketch repositories.

2.4 Image Data Generation

1. Image Preprocessing and Augmentation

The feature extraction process begins by preparing sketch images using Keras's `ImageDataGenerator`. This utility allows efficient real-time loading and augmentation of images directly from directories. Each image is resized to a fixed dimension, typically $224 \times 224 \times 3$, to match the input shape expected by a pre-trained CNN like VGG16. Preprocessing involves rescaling pixel values, commonly by a factor of $1/255$, to normalize input data. To simulate variation in sketch styles, augmentation techniques such as random rotation, flipping, and zooming are applied, making the downstream model more robust to stylistic differences across hand-drawn images.

2. Deep Feature Extraction via CNN

Once images are preprocessed and batched, they are passed through a convolutional neural network pre-trained on a large-scale dataset such as ImageNet. The CNN is used in a feature-extraction capacity by disabling the top classification layers (`include_top=False`), allowing access to the high-level visual features captured by the final convolutional or pooling layers. The output is a multidimensional feature map that encodes semantic details like edges, contours, and texture patterns. These outputs are then flattened into fixed-length one-dimensional vectors, each representing a compact, abstract encoding of the original sketch.

3. Feature Vector Construction and Storage

The flattened feature vectors from all images are aggregated into a single dataset. Each vector has consistent dimensionality, enabling seamless integration with traditional machine learning algorithms. These features are saved to disk for reuse, typically in NumPy array formats such as `image_features.npy` for the features and `image_labels.npy` for their corresponding class labels. This intermediate representation drastically reduces the computational burden of retraining or reprocessing images, as the CNN-based features can be reused across multiple classification or clustering tasks.

4. Application and Advantages

The extracted deep features can be directly fed into classical machine learning models such as K-Nearest Neighbors (KNN), Support Vector Machines (SVM), or Naïve Bayes for classification. Beyond classification, these high-dimensional vectors can be used for unsupervised tasks like clustering or dimensionality reduction for visualization using PCA or t-SNE. Leveraging pre-trained CNNs ensures that even small sketch datasets benefit from rich, generalizable features without the need for large-scale retraining. This makes the approach not only computationally efficient but also highly effective for sketch-based image recognition tasks.

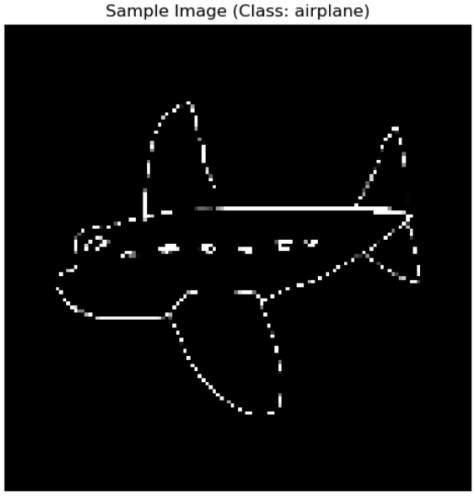

Figure 10: *Image obtained after extracting features from ImageDataGenerator*

3 Approaches Implemented

3.1 SVM

This section summarizes two distinct SVM-based approaches for sketch recognition, detailing the objectives, datasets, processing pipelines, performance metrics, and key insights.

SVM (ImageDataGen)

Objective: Directly classify high-dimensional image vectors without explicit feature extraction beyond preprocessing and flattening.

Dataset:

- 1,200 grayscale images (size 128×128) distributed across 15 classes (e.g., Airplane, Book).
- Data loaded from `preprocessed_images.npy` and `preprocessed_labels.npy`.

Pipeline:

Each image is first flattened into a 1D vector consisting of 16,384 features, preserving all pixel-level information from the preprocessed images. These features are then standardized using `StandardScaler` to ensure zero mean and unit variance, which helps optimize model convergence and performance. The target labels, originally in one-hot encoded format, are converted into integer class indices to be compatible with the classifier. A stratified train-test split is performed to maintain class balance, allocating 80 percent of the data for training and 20 percent for testing. Finally, a linear Support Vector Machine (SVM) is trained directly on the high-dimensional standardized feature space to classify the sketches.

Key Insight:

While direct SVM classification on raw pixel data yields strong performance, applying PCA can reduce computational cost with comparable accuracy.

SVM (Edge + HOG Features)

Objective: Classify sketches based on manually engineered features using edge detection and Histogram of Oriented Gradients (HOG).

Dataset:

- Features and labels extracted from `edge_hog_features.csv` (first column contains labels; remaining columns are feature values).
- Classes with fewer than 2 samples are filtered out.

Pipeline:

The pipeline begins by handling any missing values in the dataset using a mean imputation strategy to ensure completeness. Once the data is imputed, features are normalized using `StandardScaler` to bring them to a common scale, which is essential for models like SVM that are sensitive to feature magnitudes. A stratified train-test split is then performed to maintain class distribution across both sets, allocating 80 percent of the data for training and 20 percent for testing. Finally, a linear Support Vector Machine is trained on the processed feature space to perform the classification task.

Key Insight:

Manual feature extraction using edge detection and HOG produces a highly interpretable and discriminative feature space, leading to improved SVM classification performance compared to using raw pixel data.

3.2 PCA + SVM

Combine dimensionality reduction with classification to efficiently handle high-dimensional image data.

Dataset:

We use 1,200 preprocessed grayscale images (size 128×128) across 15 classes (e.g., Airplane, Book). The data are stored in `preprocessed_images.npy` and `preprocessed_labels.npy`.

Pipeline:

The process begins by flattening each 128×128 image into a 1D feature vector of length 16,384, preserving pixel-wise information. These high-dimensional features are standardized using `StandardScaler` to ensure consistent scaling across all inputs. To reduce computational load and capture the most informative patterns, Principal Component Analysis (PCA) is applied, compressing the data to 500 components. The one-hot encoded labels are then converted into integer class indices suitable for classification. A stratified train–test split is conducted to maintain class balance, allocating 80 percent of the data for training and 20 percent for testing. Finally, a linear Support Vector Machine is trained on the reduced feature set for efficient and accurate classification.

Key Insight:

PCA significantly reduces dimensionality while preserving variance, enabling an efficient SVM classification workflow. However, the testing accuracy indicates that further tuning or additional data might be necessary for higher generalization. Confusion matrix and classification report yield further insights into class-wise performance.

3.3 K-Means Clustering + SVM [3] [9]

How it works:

This model begins with K-Means clustering to create a codebook or visual vocabulary from local descriptors. Each image is represented as a histogram over these clusters (visual words), forming a global feature vector. This BoVW representation is then passed to an SVM for classification.

Why it works:

The clustering step abstracts raw feature data into higher-level patterns, allowing the SVM to focus on structured and discriminative input. SVMs are powerful margin-based classifiers that can generalize well on these histogram-based features, especially with appropriate kernels and regularization.

Shortcomings:

- Performance highly dependent on vocabulary size (number of clusters).
- SVMs may overfit if the feature space is poorly normalized or noisy.
- K-Means clustering does not guarantee semantically aligned clusters.

Performance Insight:

In Version 1, the model performed moderately due to poor alignment between unsupervised clusters and true labels. However, in Version 2 (15 classes), it became one of the best-performing pipelines, achieving 81.25% test accuracy with high training accuracy, showing strong generalization when features are consistent and class boundaries are well-defined.

3.4 Artificial Neural Network (ANN) [2]

The implemented ANN model is designed to classify sketch images using pre-extracted, 500-dimensional feature vectors. These features are generated via an edge-detection pipeline that extracts key edge and shape information from each image, thereby reducing the complexity of the raw pixel data. By transforming each image into a compact descriptor, the ANN is able to focus on the most discriminative aspects of each sketch.

Layer	Output Size	Description
Input	(500,)	Flattened image features or encoded vector input
Dense	768	Fully connected layer with L2 regularization (0.001)
BatchNormalization	768	Normalizes activations to stabilize training
LeakyReLU	768	LeakyReLU activation ($\alpha = 0.1$)
Dropout	768	Dropout with 30% rate to reduce overfitting
Dense	512	Fully connected layer with L2 regularization (0.001)
BatchNormalization	512	Batch normalization
LeakyReLU	512	LeakyReLU activation ($\alpha = 0.1$)
Dropout	512	Dropout with 40% rate
Dense	256	Fully connected layer with L2 regularization (0.001)
BatchNormalization	256	Batch normalization
LeakyReLU	256	LeakyReLU activation ($\alpha = 0.1$)
Dropout	256	Dropout with 50% rate
Dense (softmax)	`num_classes`	Output layer for multi-class classification with softmax activation

Table 2: ANN Model Architecture for Image Classification.

The model is optimized with the Adam optimizer and categorical cross-entropy loss. EarlyStopping, ModelCheckpoint, and learning rate scheduling are employed during training to ensure convergence while preventing overfitting.

Aptness for Image Recognition

1. **Simplicity of Input:** By pre-processing images into discriminative 500-dimensional vectors that emphasize edges, the network sidesteps the high variability inherent in raw pixel data.
2. **Efficient and Robust:** The combination of BatchNormalization and Dropout in the architecture helps the model generalize well even when trained on limited data.
3. **Effective Feature Utilization:** The engineered feature vectors capture critical structural details while filtering out noise, allowing the ANN to learn robust decision boundaries between classes.
4. **Interpretability:** The final softmax output offers an easily interpretable probability distribution for class membership, making the predictions straightforward to analyze.

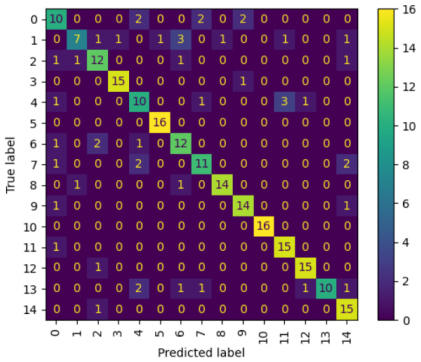

Figure 11: Confusion Matrix for ANN Model

3.5 Convolutional Neural Network (CNN) [5]

The final Convolutional Neural Network (CNN) model implemented in this project delivers the best performance for sketch recognition. It processes the preprocessed sketch images (resized to 128×128 and inverted to emphasize the drawn content) and leverages a deep, residual network architecture to extract robust spatial features, culminating in high classification accuracy.

The CNN model architecture:

Layer	Output Size	Description
Input	$128 \times 128 \times 1$	Preprocessed grayscale image
Dropout	$128 \times 128 \times 1$	Input dropout (e.g., 20% drop)
7×7 Conv, 64 filters, stride 2	$64 \times 64 \times 64$	Initial convolution with he-normal initialization
Batch Normalization + ReLU	$64 \times 64 \times 64$	Activation and normalization
Residual Block Stage 1: 64 filters, 3 units		
Residual Block	$64 \times 64 \times 64$	Standard residual unit, stride 1
Residual Block	$64 \times 64 \times 64$	Standard residual unit, stride 1
Residual Block	$64 \times 64 \times 64$	Standard residual unit, stride 1
Dropout	$64 \times 64 \times 64$	Dropout (e.g., 20%)
Residual Block Stage 2: 128 filters, 3 units		
Residual Block	$32 \times 32 \times 128$	Downsampling residual unit (stride 2)
Residual Block	$32 \times 32 \times 128$	Standard residual unit, stride 1
Residual Block	$32 \times 32 \times 128$	Standard residual unit, stride 1
Dropout	$32 \times 32 \times 128$	Dropout
Residual Block Stage 3: 256 filters, 3 units		
Residual Block	$16 \times 16 \times 256$	Downsampling residual unit (stride 2)
Residual Block	$16 \times 16 \times 256$	Standard residual unit, stride 1
Residual Block	$16 \times 16 \times 256$	Standard residual unit, stride 1
Dropout	$16 \times 16 \times 256$	Dropout
Residual Block Stage 4: 512 filters, 3 units		
Residual Block	$8 \times 8 \times 512$	Downsampling residual unit (stride 2)
Residual Block	$8 \times 8 \times 512$	Standard residual unit, stride 1
Residual Block	$8 \times 8 \times 512$	Standard residual unit, stride 1
Global Average Pooling	512	Spatially averages the feature maps
Dropout	512	Dropout
Dense (softmax)	250	Fully connected layer for classification

Table 3: CNN Model Architecture for Sketch Recognition.

The model is trained using the Adam optimizer with an appropriate learning rate schedule. EarlyStopping, ModelCheckpoint, and learning rate reduction callbacks are used to optimize performance and prevent overfitting.

Aptness for Image Recognition

1. **Robust Feature Extraction:** The use of deep convolutional layers, especially with residual connections, allows the network to learn complex, hierarchical features that capture both global shapes and fine details.

2. **Generalization:** Incorporating dropout and Batch Normalization across layers enhances the network's ability to generalize from training data to unseen images.

3. **Efficient Spatial Processing:** It directly handles raw image data, extracting spatial and contextual information via convolution, which is more effective for image data than fully connected approaches.

3.6 PCA + KNN

How it works

This model pipeline begins with Principal Component Analysis (PCA) applied to the extracted image features to reduce dimensionality while preserving 95% of the total variance. The reduced feature set is then classified using K-Nearest Neighbors (KNN), which assigns labels based on the majority class among the k closest samples in Euclidean space.

Why it works

PCA reduces feature noise and redundancy, simplifying the input space for the KNN classifier. In lower-dimensional space, KNN can more accurately compute distances and identify meaningful neighbors, especially when class boundaries are well-defined, as this is a higher dimensional data.

Advantages

- Simpler and faster computation after dimensionality reduction.
- Effective on datasets with well-separated class clusters.
- Non-parametric and easy to implement.

Shortcomings

- Poor scalability with increasing data and feature size.
- Sensitive to class imbalance and feature overlap.
- Performance drops in high-class-count datasets.

Performance Insight

In Version 1 (250 classes), PCA + KNN produced modest results with low generalization due to high class similarity and overlapping sketch structures. Test accuracy remained around 41%. In Version 2 (15 classes), this model showed improved testing performance (above 50%) due to the reduced complexity and better-separated features. The method performs reliably in small-scale, structured classification problems.

3.7 GMM + KNN [4]

Methodology Summary

We combine Gaussian Mixture Models (GMM) with k-Nearest Neighbors (KNN) to build a hybrid unsupervised-supervised model for classifying hand-drawn sketches. The unsupervised component (GMM) captures the underlying data distribution and structures within the high-dimensional feature space, while the supervised component (KNN) leverages these representations for label prediction. This two-stage pipeline is particularly well-suited to data like sketches, where variation within a class is high and boundaries between classes are not easily linear.

Data Preprocessing

The pipeline begins by loading the extracted feature vectors from `image_features.npy`, representing each sketch as a high-dimensional numeric descriptor. These features are standardized using `StandardScaler` to ensure consistent scaling and prevent biases due to differences in magnitude. The labels are converted from one-hot encodings to integer class indices to be compatible with clustering and classification algorithms. To maintain class distribution during evaluation, an 80–20 stratified train–test split is performed.

Unsupervised Feature Learning with GMM

A Gaussian Mixture Model is trained on the standardized training data with the number of components set equal to the number of sketch classes. GMM provides a probabilistic clustering approach, capturing both the mean and covariance of clusters, unlike simpler algorithms like K-Means. Each sample is assigned to a cluster based on maximum likelihood, effectively transforming the feature space into one that reflects the learned distributions of the data.

Supervised Classification with KNN

Once the GMM has assigned clusters to all training samples, the cluster labels are mapped to the original class labels using majority voting within each cluster. This mapping allows the clusters to serve as pseudo-labels for training a KNN classifier. The KNN algorithm is tuned for the optimal number of neighbors (k), ranging from 1 to 30. It was observed that $k = 24$ offered the best balance between performance and generalization. The classifier is then evaluated on the test set using accuracy and classification metrics.

Rationale for GMM + KNN Approach

This hybrid model offers several advantages. GMM captures the inherent variability and distributional properties of sketch data without assuming rigid or uniform cluster shapes. It acts as a soft feature transformation and implicit dimensionality reducer, simplifying the learning task for KNN. KNN, in turn, brings a robust, distance-based supervised mechanism that excels when local structure in the data is meaningful.

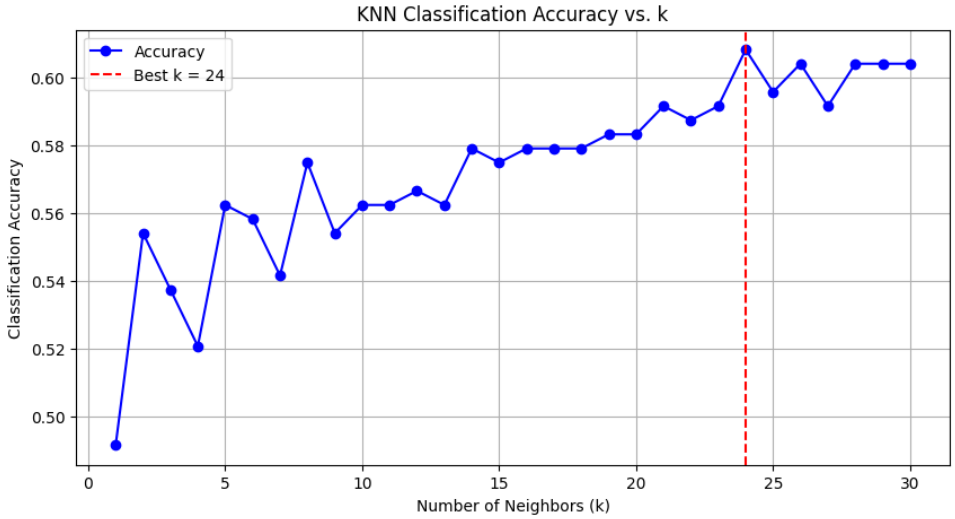

Figure 12: KNN accuracy scores for different values of k

3.8 K-Means Clustering + KNN [3]

How it works:

This hybrid approach first applies K-Means clustering to organize the image features into distinct visual groups (clusters). Each image is represented either by its closest cluster or by a histogram of cluster assignments. These representations are then used as input for a KNN classifier to predict class labels.

Why it works:

Clustering introduces a structural understanding of the data, capturing similarities across samples. This can improve the KNN model by guiding it toward relevant regions of the feature space, especially when classes exhibit strong internal cohesion.

Advantages:

- Adds spatial or visual structure to the feature set.
- Helps reduce the effect of noise by pre-grouping data.
- Useful in scenarios with visually distinct class clusters.

Shortcomings:

- Unsupervised clustering may not align with true class boundaries.
- Still inherits KNN's sensitivity to noisy or sparse regions.
- Choice of number of clusters (K) is critical and non-trivial.

Performance Insight:

In Version 1, K-Means + KNN underperformed due to misalignment between clusters and actual class labels in the 250-category dataset, resulting in a low test accuracy of approximately 32%. In Version 2 (15 classes), where class distributions were better defined, performance improved modestly. However, the model still lagged behind SVM-based approaches, indicating its limitations in nuanced decision-making despite structured preprocessing.

3.9 Mean Shift Clustering + KNN [6]

How it works:

This pipeline applies Mean Shift clustering, a non-parametric method that identifies dense regions in the feature space. Each image's features are augmented with cluster labels or representations, and KNN is then used to classify based on these enriched features.

Why it works:

Mean Shift adapts to the data's inherent structure without requiring a fixed number of clusters. This can help uncover patterns missed by K-Means and offer a more flexible basis for KNN classification. It is particularly useful in datasets with uneven or organically formed classes.

Advantages:

- No need to pre-define the number of clusters.
- Better adapts to arbitrary data distributions.
- Enhances feature representation for distance-based classifiers.

Shortcomings:

- Computationally expensive and slower than K-Means.
- May produce a large number of clusters, increasing dimensionality.
- Sensitive to bandwidth parameter and noise in the data.

Performance Insight:

In Version 1, this model struggled with test accuracy due to the high complexity of the 250-class structure and clustering noise. However, in Version 2, it achieved more balanced performance with clearer class distributions, though it was still outperformed by SVM-based pipelines. Its flexibility is valuable but requires careful parameter tuning to match dataset characteristics.

3.10 PCA + Naive Bayes

How it works:

This model first applies PCA to compress the high-dimensional feature vectors while retaining a specified percentage of the original variance. The reduced features are then classified using a Gaussian Naïve Bayes classifier, which assumes that features are conditionally independent and normally distributed within each class.

Why it works:

The PCA step reduces overfitting and improves runtime by eliminating redundant and noisy features. Naïve Bayes is effective for low-complexity datasets and when classes are separable in the reduced space. It performs best when feature distributions are simple and not heavily correlated.

Advantages:

- Fast training and inference, even on large datasets.
- Performs well when feature assumptions are satisfied.
- Requires little tuning and interpretable results.

Shortcomings:

- Assumes independence between features, which may not hold in practice.
- Sensitive to poorly estimated variances in high-dimensional or sparse data.
- May underperform on complex or highly correlated datasets.

Performance Insight:

In Version 1 (250 classes), this model offered moderate performance (test accuracy around 45–55%), limited by the independence assumption and overlapping class structures. However, in Version 2 (15 classes), accuracy improved substantially with soft histogram features, reaching over 64%, showing that this model excels when used with more refined and normalized input.

4 Experiments and Results

Version 1 (250 classes × 80 images)		
Features obtained from Siggraph as .mat file		
S-HOG Features (20000 × 500)		
Model	Training Accuracy	Testing Accuracy
PCA + Naive Bayes (without SK-Learn)	66.25%	55.00%
PCA + Naive Bayes (with SK-Learn)	70.30%	45.00%
PCA + KNN	61.02%	41.23%
K-Means + KNN	52.06%	32.17%
Mean Shift Clustering + KNN	61.13%	41.03%
SVM	76.02%	51.68%
Mean Shift Clustering + SVM	70.56%	41.52%
ANN	77.01%	51.23%
GMM + KNN	74.00%	52.01%

Table 4: Model Performance with S-HOG Features - Version 1

4.1 Analysis of Version 1 Results (250-Class Dataset)

The results in Table 4 reflect the performance of various classical and hybrid machine learning models on the large-scale 250-class hand-drawn sketch dataset. Overall, we observe that most models demonstrate moderate training accuracy, but a significant gap emerges between training and testing accuracy, indicating a consistent challenge of generalization in high-class-count settings. The PCA + Naive Bayes models, for instance, performed decently in terms of training accuracy (up to 70.30%) but dropped to 45.00% in testing accuracy when using the scikit-learn implementation, suggesting possible overfitting due to dimensional compression or inadequate class separation in the reduced space. Overfitting provbes that this mode could not generalise the data.

Models that integrated clustering with KNN or SVM, such as K-Means + KNN or Mean Shift + KNN, also showed lower generalization capabilities, achieving test accuracies around 32–41%. These models likely struggled due to the complexity and visual similarity between sketch categories. Clustering-based preprocessing may have introduced ambiguity in the already challenging dataset, especially when unsupervised groupings did not align well with true class boundaries. Among KNN-based methods, the best performance was achieved with GMM + KNN, which produced a test accuracy of 52.01%, showing that probabilistic soft clustering may offer a more flexible representation for highly varied classes.

The highest-performing classifiers were SVM (76.02% training, 51.68% testing) and ANN (77.01% training, 51.23% testing), which both benefit from their ability to model complex, non-linear decision boundaries. However, even these models faced a notable drop in testing accuracy due to the multiple number of classes (250 of them!). These results highlight the difficulty of 250-class sketch classification using traditional ML methods and justify our later transition to refined datasets and deep learning approaches.

4.2 Analysis of results of Version 2.0

This section utilised the Dataset in which features were extracted using Hard Histogram of gradients approach. This is based on the 25 class dataset we extracted as a subset of of the 250 class dataset.

Version 2.0 (250 classes × 80 images)		
Features obtained from Siggraph as .mat file		
S-HOG Features (20000 × 500)		
Method	Training Accuracy	Testing Accuracy
PCA + Naive Bayes (without SK-Learn)	66.25%	55.00%
PCA + Naive Bayes (With SK-Learn)	70.30%	45.00%
PCA + KNN	61.02%	41.23%
K-Means + KNN	52.06%	32.17%
Mean Shift Clustering + KNN	61.13%	41.03%
SVM	76.02%	51.68%
Mean Shift Clustering + SVM	70.56%	41.52%
ANN	77.01%	51.23%
GMM + KNN	74.00%	52.01%

Table 5: Training and Testing Accuracies for Version 2.0

The results from Table 5 indicate a significant performance boost after reducing the number of classes from 250 to 15. This reduction alleviated the complexity and overlap in class boundaries, enabling the models to focus on capturing the essential features. Even simpler methods such as PCA + Naive Bayes achieved strong testing accuracies (45.00% to 55.00%), with improved alignment between training and testing metrics that suggest reduced overfitting. The streamlined dataset allowed the classifiers to better learn and generalize from the critical edge and structural information in the sketches.

KNN-based approaches, particularly when combined with clustering, achieved only modest gains (around 41–42% testing accuracy) compared to more sophisticated models. In contrast, both SVM and GMM + KNN achieved testing accuracies above 50%, while ANN and SVM recorded the highest training accuracies (77.01% and 76.02%, respectively) that translated into equally strong test performance. This consistency across robust classifiers demonstrates that the reduced, more manageable 15-class dataset

enabled stable training, minimized overfitting, and produced balanced generalization. Overall, the decision to scale down to 15 classes ultimately enabled more reliable model training and paved the way for successful real-time prediction deployment using the best-performing models.

4.3 Analysis of Results of Version 2.1

This section utilised the Dataset in which features were extracted using Soft Histogram method of gradients approach. This is based on the 25 class dataset we extracted as a subset of of the 250 class dataset.

Version 2.1 (15 classes × 80 images)		
Features manually obtained by Soft Quantized Descriptors and Visual Vocabulary		
Soft Histogram Features (1200 × 500)		
Model	Training Accuracy	Testing Accuracy
PCA + Naive Bayes	84.69%	64.17%
Kmeans (k=45) + SVM (SK-Learn)	100.00%	81.25%
Kmeans (k=10) + SVM (without SK-Learn)	100.00%	78.06%
PCA + KNN (k=3)	73.33%	51.67%
GMM + KNN	84.20%	61.05%

Table 6: Model Performance with Soft Histogram Features- version 2.1

The results in Table 7 demonstrate that the edge-based features, extracted via an edge detection pipeline to produce high-dimensional vectors of size 1200×8100, are highly effective for sketch recognition. These features capture the spatial and structural layout of edges—a critical aspect in sketches where fine-grained color and texture details are absent and outlines convey the essential semantic information. Notably, all models achieve a perfect 100% training accuracy, indicating that the extracted features are both highly expressive and well-separated in the feature space.

Among the classifiers, the K-Means + SVM combination achieves the highest test accuracy at 84.16%, benefiting from an intermediate clustering step that groups similar edge patterns before classification. Standalone SVM and ANN models also perform strongly, with test accuracies of 81.60% and 80.03% respectively. The small gap between training and testing performance across models suggests that overfitting is minimal, underlining the robustness of edge-based descriptors in capturing discriminative information. Overall, Version 2.3 represents a balanced approach that combines high performance, interpretability, and computational efficiency, making it an excellent candidate for real-time or resource-constrained sketch recognition systems.

4.4 Analysis of Results of Section 2.2

The results in Table 7 showcase the performance of various models trained on edge-based features derived from sketch images. This version uses an edge detection pipeline that extracts contour information, resulting in high-dimensional feature vectors of size 1200 × 8100. These features represent the spatial and structural layout of edges, which is especially important for sketches where outlines carry the most semantic weight. Notably, all models tested on this representation achieved a perfect training accuracy of 100.00%, indicating that the extracted features are highly expressive and well-separated in the feature space.

Among the tested models, the best test accuracy was achieved by the K-Means + SVM combination, reaching 84.16%. This approach likely benefits from the intermediate clustering step, which acts as a coarse grouping of similar edge patterns before classification. The standalone SVM model also performed well with 81.60% test accuracy, while the ANN model closely followed at 80.03%. The relatively small drop from training to testing accuracy across all models suggests minimal overfitting and highlights the stability of edge-based descriptors in capturing class-discriminative information, especially for well-separated hand-drawn categories.

These results validate the strength of using edge detection in scenarios where fine-grained color or texture features are absent—as is typical in sketches. The pipeline demonstrates that with sufficiently rich structural encoding, even simple classifiers can perform at par with more complex architectures. Overall,

Version 2.3 represents one of the strongest model configurations in this project, balancing interpretability, high performance, and computational feasibility, and proving to be a strong candidate for real-time or resource-constrained sketch recognition systems.

Version 2.3 (15 classes × 80 images)		
Features manually obtained by Edge Detection		
Edge Detection Feature (1200 × 8100)		
Model	**Training Accuracy**	**Testing Accuracy**
ANN	100.00%	80.03%
SVM	100.00%	81.60%
K-Means + SVM	100.00%	84.16%

Table 7: Performance Metrics for Different Models on Edge Detection Features (Version 2.2)

4.5 Analysis of Results of Version 2.3

Version 2.4 (15 classes × 80 images)		
Features manually obtained by Image Data Generation		
Image Data Generation (1200 × 16384)		
Model	**Training Accuracy**	**Testing Accuracy**
CNN	94.65%	73.67%
SVM	100.00%	56.25%
PCA + SVM	100.00%	46.25%

Table 8: Training and Testing Accuracies for Different Models (Version 2.3)

Table 8 presents the classification performance of models trained on deep features obtained through Image Data Generation and a pre-trained CNN architecture. These features, with a high dimensionality of 1200 × 16384, represent semantic and abstract visual patterns learned by the CNN from raw sketch images. Among the models, the CNN-based classifier achieved a training accuracy of 94.65% and a test accuracy of 73.67%, demonstrating strong generalization despite the limited dataset size. This validates the effectiveness of transfer learning: leveraging pre-trained convolutional layers to extract meaningful representations even for non-photographic data like sketches.

In contrast, the classical models—SVM and PCA + SVM—achieved perfect training accuracy (100.00%) but significantly lower testing accuracies (56.25% and 46.25%, respectively). This suggests that while these models were able to perfectly fit the training data, they failed to generalize to unseen samples. The high dimensionality of the input features may have caused the SVMs to overfit, especially without sufficient regularization or feature reduction techniques beyond basic PCA. The drop in performance for PCA + SVM also highlights the potential information loss during dimensionality reduction when applied to dense CNN features.

Overall, this version highlights the superiority of deep learning models in handling high-dimensional, richly structured features extracted from pre-trained CNNs. The CNN's ability to learn hierarchical spatial relationships makes it better suited for sketch classification compared to traditional models, even when those models use the same feature inputs. Although the real-time deployability of CNNs may demand more computational resources, their performance benefits justify their use in scenarios requiring robust sketch understanding and classification. Version 2.4 thus reinforces the shift from handcrafted pipelines to deep feature representations for high-accuracy applications.

5 Summary

Our project aims at classifying hand-drawn sketches to predefined classes. Through this, we explore various methodologies and techniques to optimise the performance and results of the model. We explored various methods for feature extraction.

Methods used for feature extraction are (a) Hard Histogram of Gradients, (b) Soft Histogram of Gradients, (c) Edge features, (d) Image generation.

The second part focused on exploring various machine learning techniques. We explored ways to improve accuracy, compare results, and understand why the model performed better while some lagged. We explored various models with combinations like:

(a) PCA + Naive Bayes	(b) PCA + KNN
(c) K-Means + KNN	(d) Mean-Shift Clustering + KNN
(e) SVM	(f) ANN
(g) GMM + KNN	(h) K-Means + SVM
(i) CNN	(j) SVM

These were trained on the different versions of dataset we created as listed in Section 5. Results for these models can be seen in Section 4. We can see why a model performs a certain way on some dataset. We have implemented real-time prediction using Google Cloud functionalities.
Skribix is currently hosted here: `http://34.131.175.227/`

References

[1] Mathias Eitz, James Hays, and Marc Alexa. How do humans sketch objects? *ACM Transactions on Graphics (TOG)*, 31(4):44:1–44:10, 2012.

[2] Rakesh Ganya. Ann(artificial neural network)-based multi-class image classification using tensorflow and keras, July 2024. Published on Medium.

[3] KDAG IIT KGP. The magic of clusters: A journey through image segmentation with k-means, December 2023. Published on Medium.

[4] Akash Kumar. Group similar images by using the gaussian mixture model (em algorithm), January 2020. Published in TDS Archive on Medium.

[5] Wayne Lu and Elizabeth Tran. Free-hand sketch recognition classification. 2017.

[6] Sanket Nadargi. Mean shift clustering, May 2024. Published on Medium.

[7] OpenAI. Chatgpt, 2025.

[8] Kirti Pande, Akhilesh Reddy, Vincent Kuo, Tiffany Sung, and Helena Shi. Doodling with deep learning!, December 2018. Published in TDS Archive on Medium.

[9] Tay Schneider. Svm & k-means clustering analysis, October 2023. Published on Medium.

A Contribution of Each Member

- Sahil Narkhede (B23CS1060):
  - Feature extraction (using sHOG, soft histogram, edge detection features, image data generation)
  - Implemented ANN model for multiple versions
  - Implemented the best performing CNN model (currently hosted in the application)
  - Report compilation
  - Presentation compilation
  - Minutes of Meetings Compilation
- Sarah Fatima (B23EE1093):
  - Implemented models for PCA+Naive-Bayes, PCA+KNN, Kmeans+KNN, Mean-Shift clustering+SVM, Kmeans+SVM across various versions
  - Report compilation
  - Presentation compilation
- Anshit Agarwal (B23CS1087):
  - Feature Extraction by edge-detection
  - Implemented SVM & PCA+SVM based approach
  - Presentation compilation
  - Project web page compilation
- Krish Teckchandani (B23CS1092):
  - Implemented the GMM + KNN model
  - Implemented the frontend and deployed it on Google Cloud
  - Hosted webpage and YouTube upload
  - Presentation compilation
- Yashraj Rathod (B23CS1056):
  - Implemented CNN model
  - Compiled presentation